Name: Atharva Waghmode
Roll No. - 59
Div - D15A

Experiment 7

Aim: Experiment to implement any one of the clustering algorithms using Rapid Miner and Python.

Theory :

Clustering groups observations that share similar features without relying on pre-assigned labels. As an unsupervised learning method, it helps reveal structure in a dataset. K-Means divides records into k groups, DBSCAN finds dense regions and treats isolated points as noise, and hierarchical clustering builds nested groups. These methods are used in customer segmentation, anomaly detection, image analysis, and bioinformatics. The best choice depends on the data shape, noise level, and intended analysis. Clustering is useful when the goal is to explore a dataset rather than predict a known class. The groups can then be studied to understand shared patterns within the records.

Clustering can also be viewed as a way to discover natural groups when no target categories are supplied. Records with similar characteristics are placed together, while dissimilar records are assigned to other groups.

- Discover the natural grouping or structure in unlabelled data without predefined categories.
- Data points are assigned to clusters based on similarity or distance measures.
- Uses Euclidean distance, cosine similarity or other metrics depending on data type and clustering method.

Types of Clustering

1. Hard Clustering

Hard clustering assigns each data point to exactly one cluster. A data point cannot belong to multiple clusters, making the grouping clear and easy to interpret.

- Each data point belongs to only one cluster
- No overlap between clusters
- Simple and easy to interpret

2. Soft Clustering

Soft clustering allows a data point to belong to multiple clusters with different probabilities. Instead of assigning a strict cluster, it gives a degree of membership to each cluster.

Example

A data point may belong 70% to Cluster 1 and 30% to Cluster 2, indicating that it shares characteristics with both groups.

Use Cases

- **Overlapping class boundaries:** Useful when data points cannot be clearly separated into distinct groups.
- **Customer personas:** Helps represent customers who share traits with multiple behavioral groups.
- **Medical diagnosis:** Patients may show symptoms related to multiple conditions.

Benefits

- **Captures ambiguity:** Represents uncertainty when cluster boundaries are not clear.
- **Models gradual transitions:** Allows smooth transitions between clusters instead of strict separation.

K-Means Clustering:

K-Means Clustering groups similar data points into clusters without needing labeled data. It is used to uncover hidden patterns when the goal is to organize data based on similarity.

- Helps identify natural groupings in unlabeled datasets
- Works by grouping points based on distance to cluster centers
- Commonly used in customer segmentation, image compression and pattern discovery
- Useful when you need structure from raw, unorganized data.

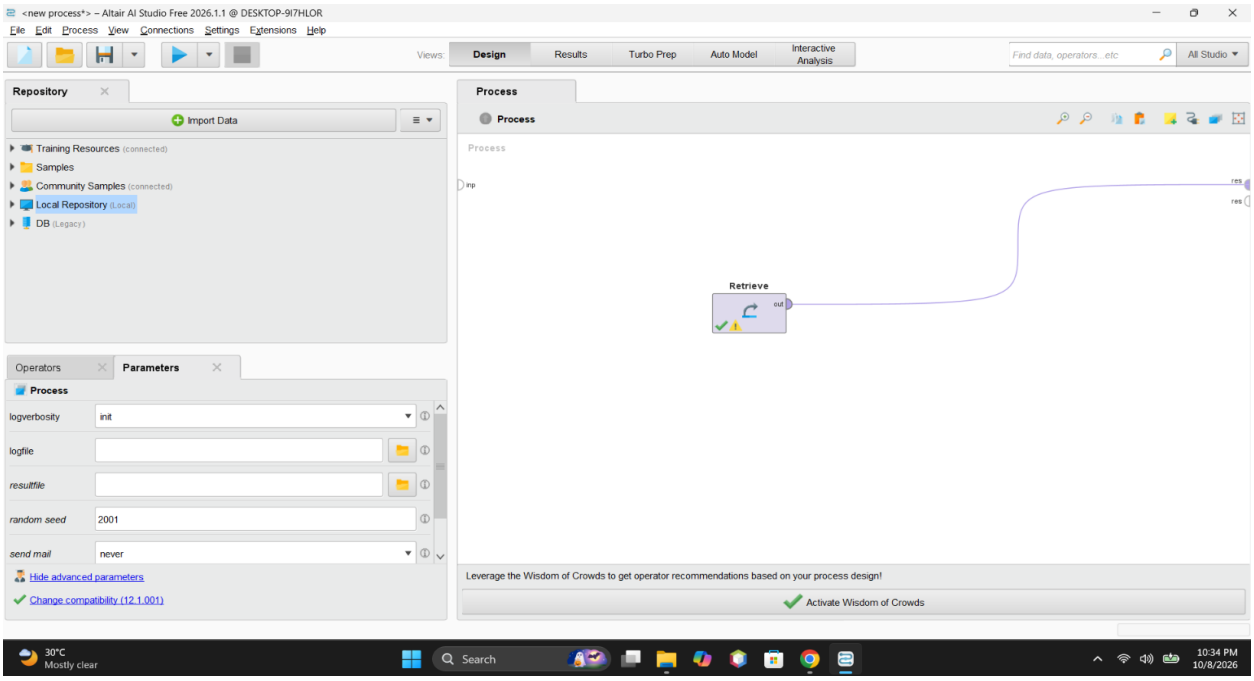
Uses of K-Means Clustering

K-Means is popular in a wide variety of applications due to its simplicity, efficiency and effectiveness. Here's why it is widely used:

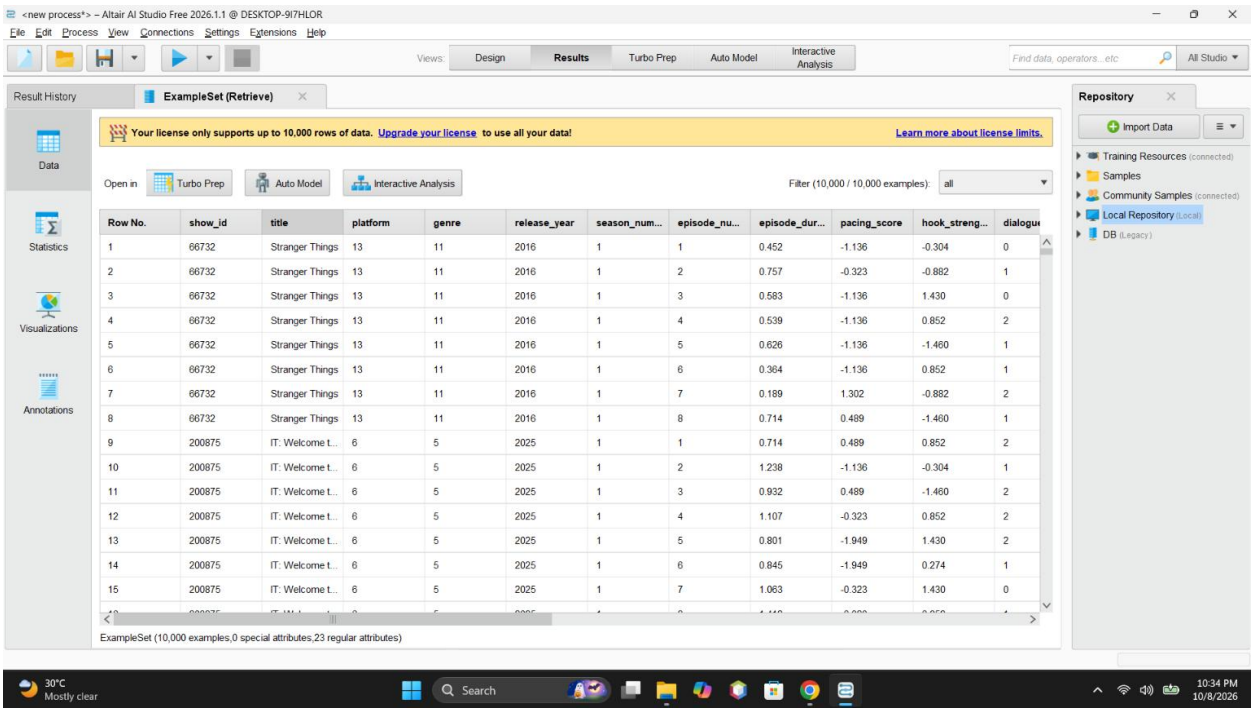
1. **Data Segmentation:** One of the most common uses of K-Means is segmenting data into distinct groups. For example, businesses use K-Means to group customers based on behavior, such as purchasing patterns or website interaction.
2. **Image Compression**: K-Means can be used to reduce the complexity of images by grouping similar pixels into clusters, effectively compressing the image. This is useful for image storage and processing.
3. **Anomaly Detection:** K-Means can be applied to detect anomalies or outliers by identifying data points that do not belong to any of the clusters.
4. **Document Clustering:** In natural language processing (NLP), K-Means is used to group similar documents or articles together. It's often used in applications like recommendation systems or news categorization.
5. **Organizing Large Datasets:** When dealing with large datasets, K-Means can help in organizing the data into smaller, more manageable chunks based on similarities, improving the efficiency of data analysis.

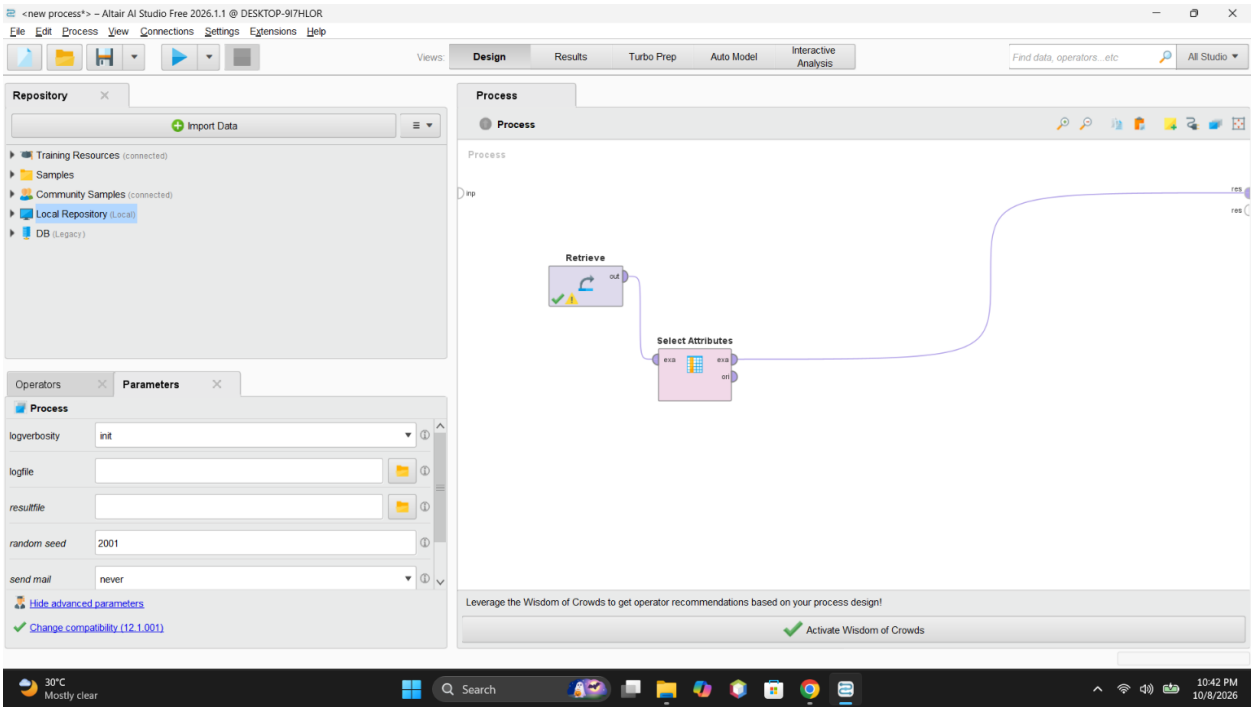
Implementation :

B. Rapid Miner

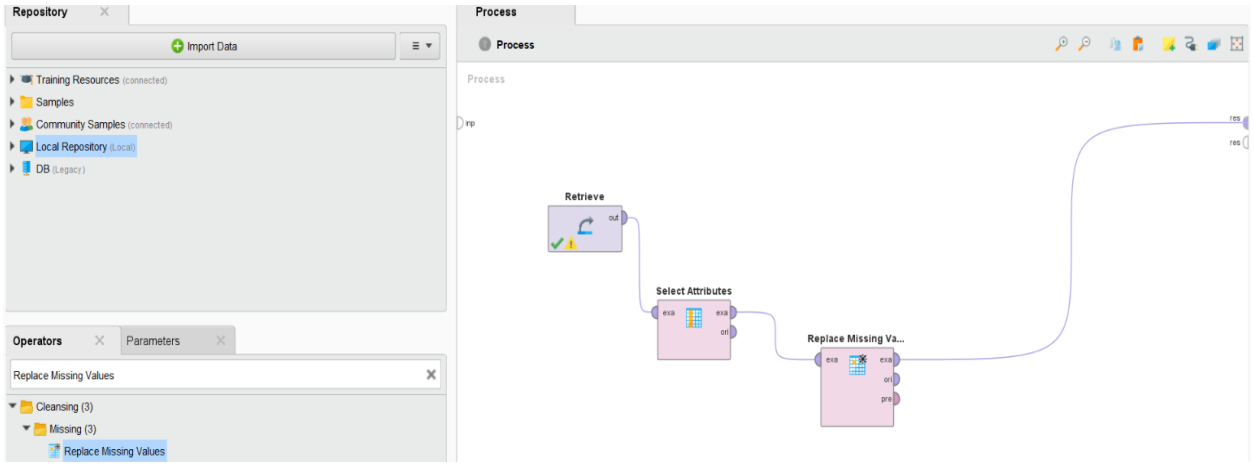


Inference: The Retrieve operator was added to the process to access the JioHotstar dataset stored in the repository. This operator loads the saved dataset into Altair AI Studio so that it can be processed and analyzed using subsequent operators.

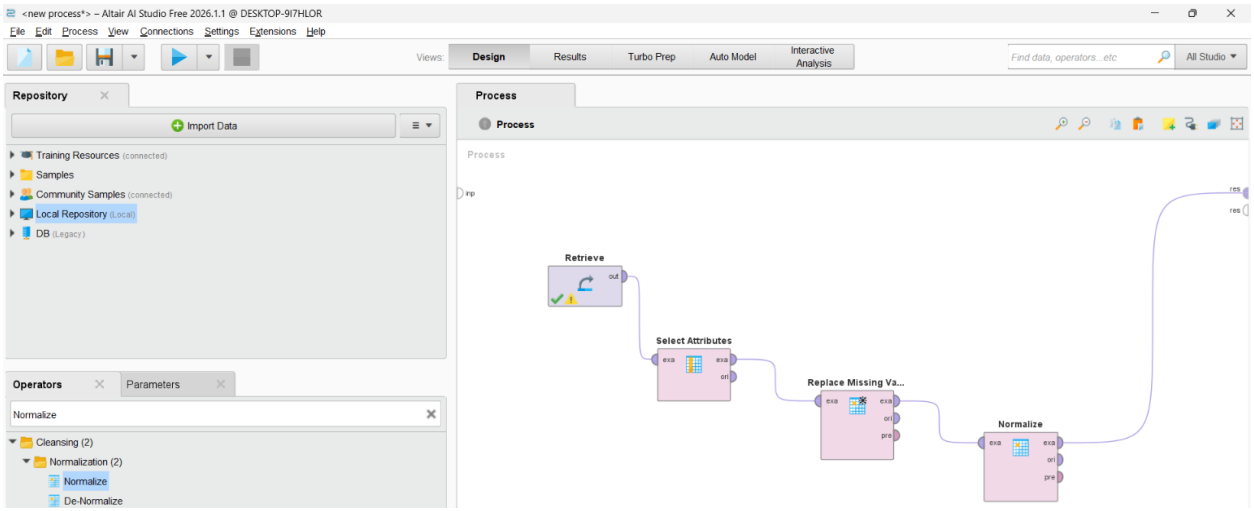




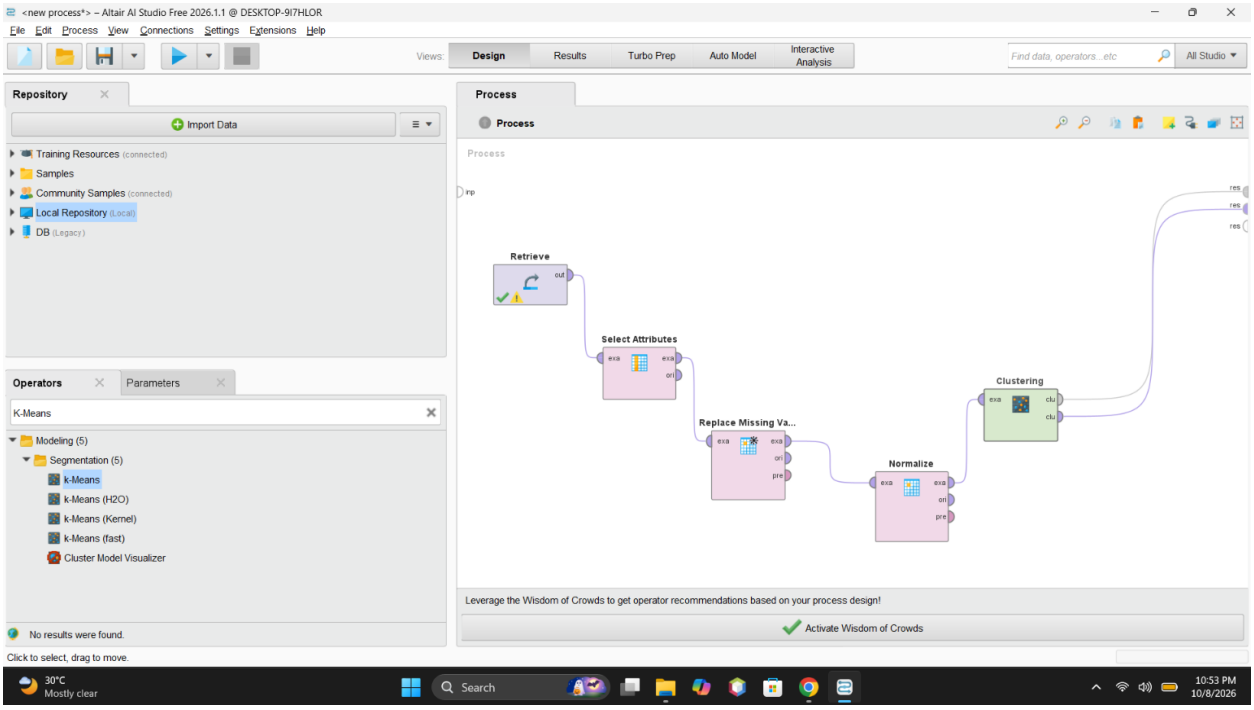
Inference: The dataset was successfully retrieved and displayed in the Results view. The Free license restricted processing to 10,000 examples, and the displayed dataset contains 10,000 rows and 23 regular attributes. The data includes show details and numerical viewing-related features required for further analysis.



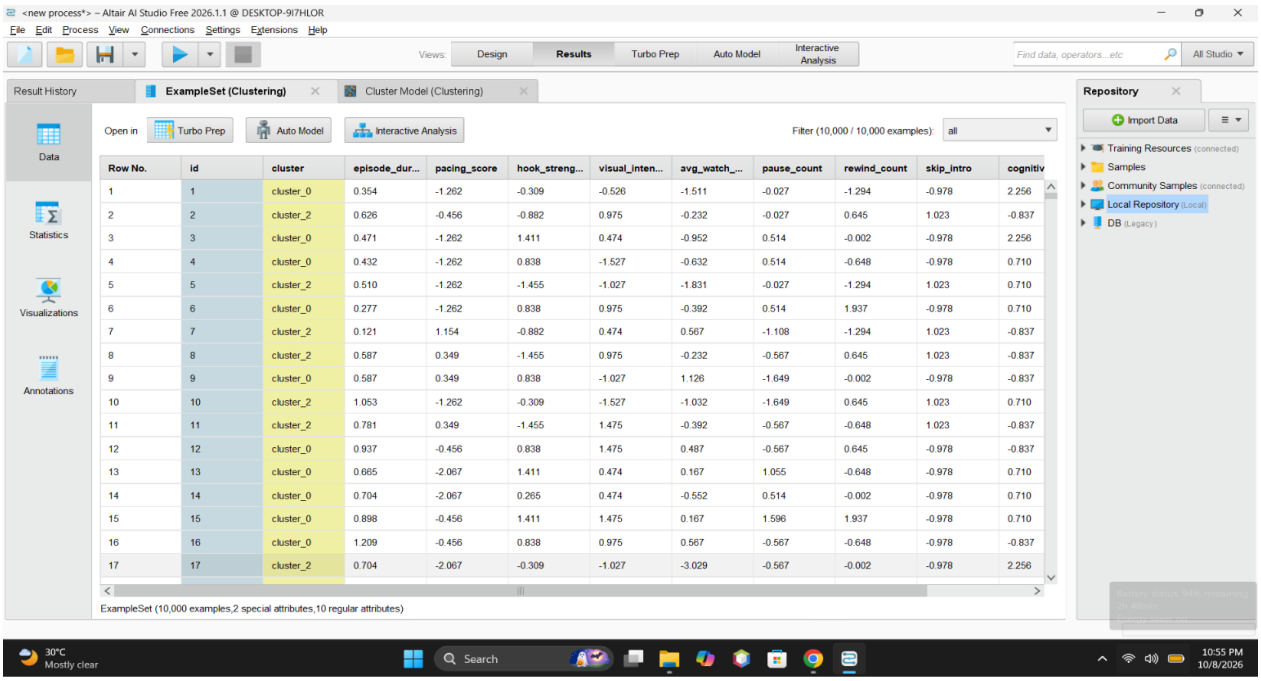
Inference: The Replace Missing Values operator was added after Select Attributes. It is used to handle missing data by replacing missing numerical values with the average of the respective attributes. This improves data completeness before normalization and clustering.



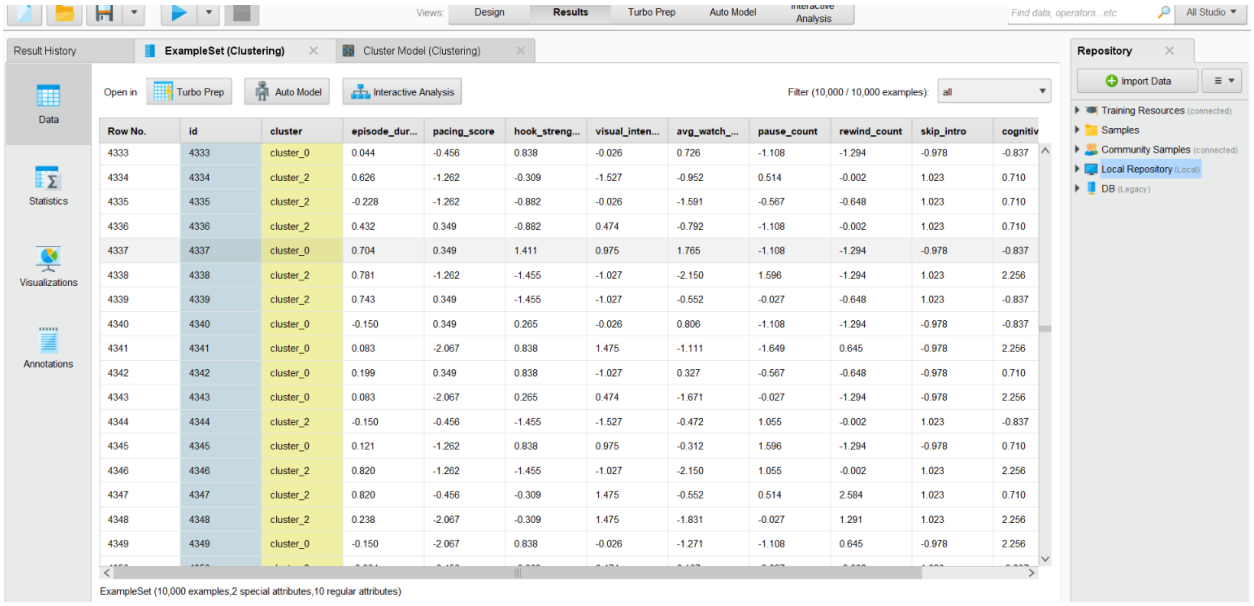
Inference: The Normalize operator was connected after Replace Missing Values to standardize the selected numerical features using Z-transformation. This makes the features comparable in scale and prevents attributes with larger numerical ranges from dominating the distance calculations performed by K-Means.



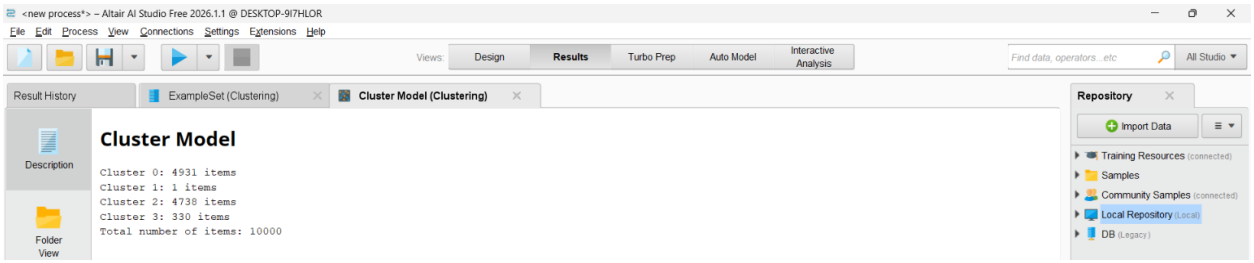
Inference: The K-Means clustering operator was added to the workflow after data preparation. The process connects Retrieve, Select Attributes, Replace Missing Values, Normalize, and Clustering in sequence. The prepared dataset is passed to K-Means to group similar records into four clusters and analyze different viewing-behaviour patterns.



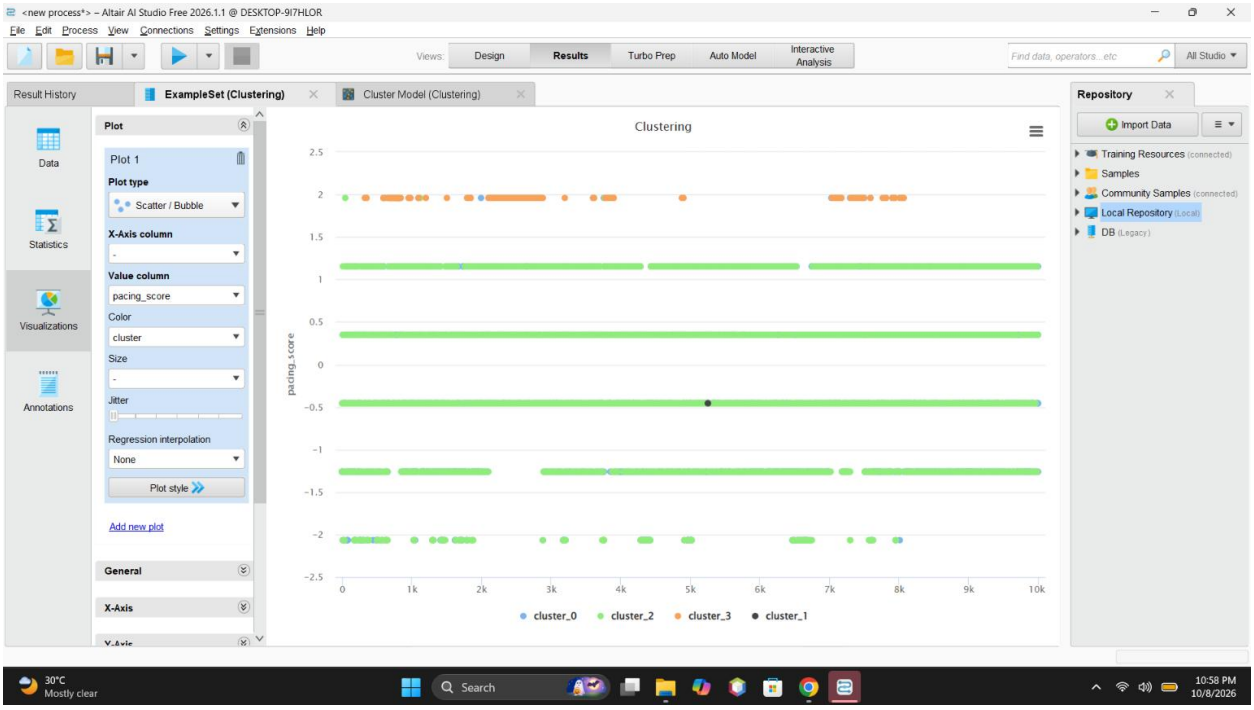
After executing the process, Altair AI Studio displays the resulting ExampleSet containing the assigned cluster for each record. The cluster column indicates which cluster each record belongs to, such as cluster_0 or cluster_2. The numerical attribute values are standardized values because the Normalize operator uses Z-transformation. The results show that the records have been grouped according to similarities in their selected attributes.



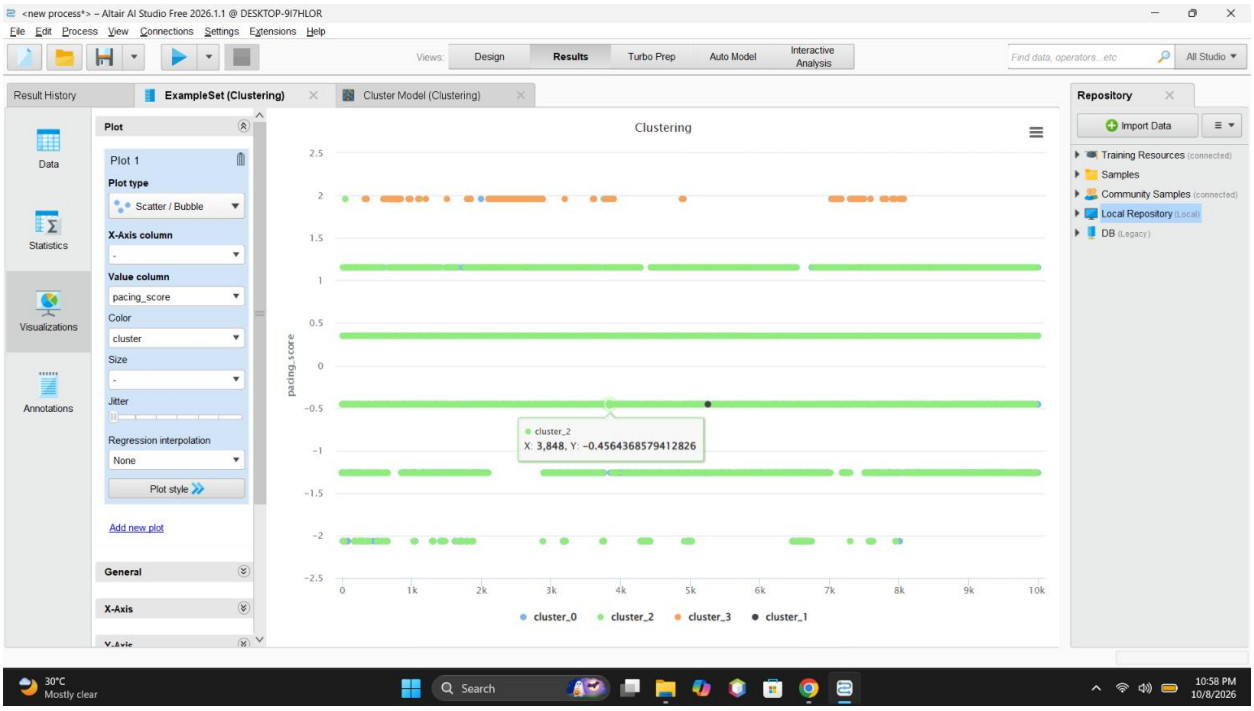
This screenshot shows another portion of the ExampleSet results table. Each row represents a record from the dataset, and the cluster column shows its assigned group. Viewing multiple rows helps verify that the clustering operator has assigned cluster labels throughout the processed dataset. The displayed values are normalized values rather than the original measurements.

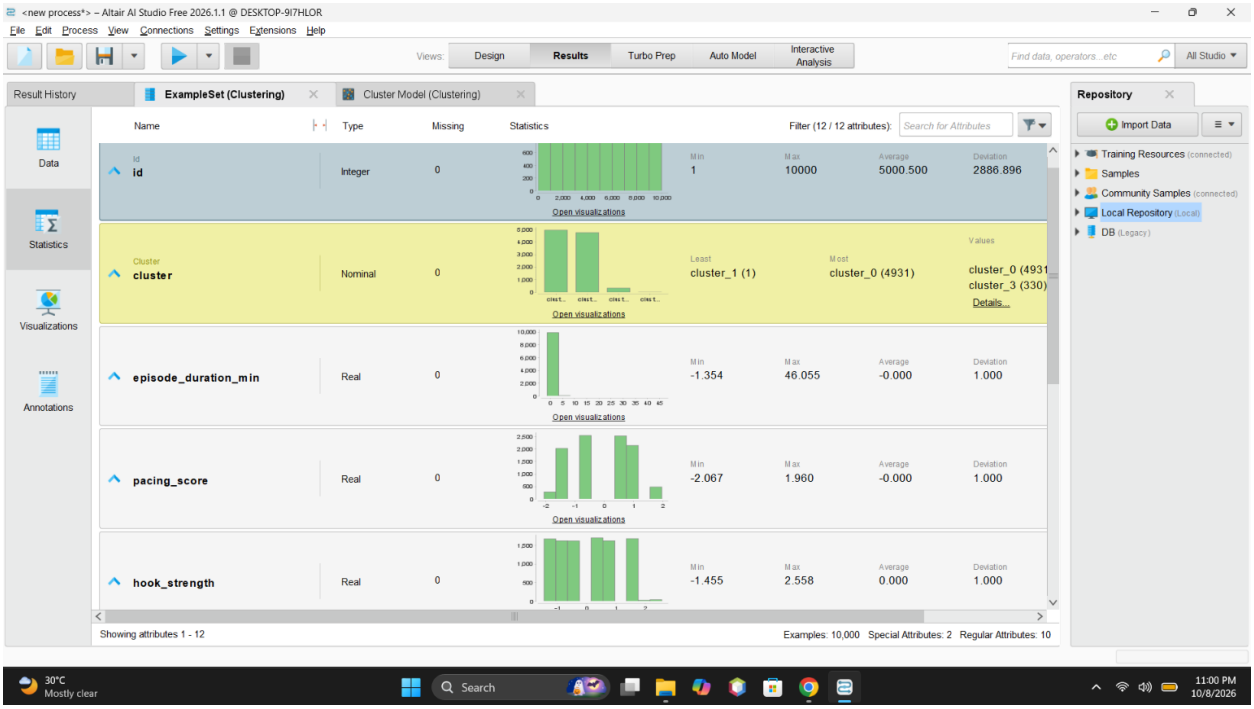


The Cluster Model displays the number of records assigned to each cluster. The output shows that Cluster 0 contains 4,931 records, Cluster 1 contains 1 record, Cluster 2 contains 4,738 records, and Cluster 3 contains 330 records. The total number of processed records is 10,000. The results indicate that most records belong to Clusters 0 and 2, while Cluster 1 contains only one record, which may represent an unusual observation or outlier.

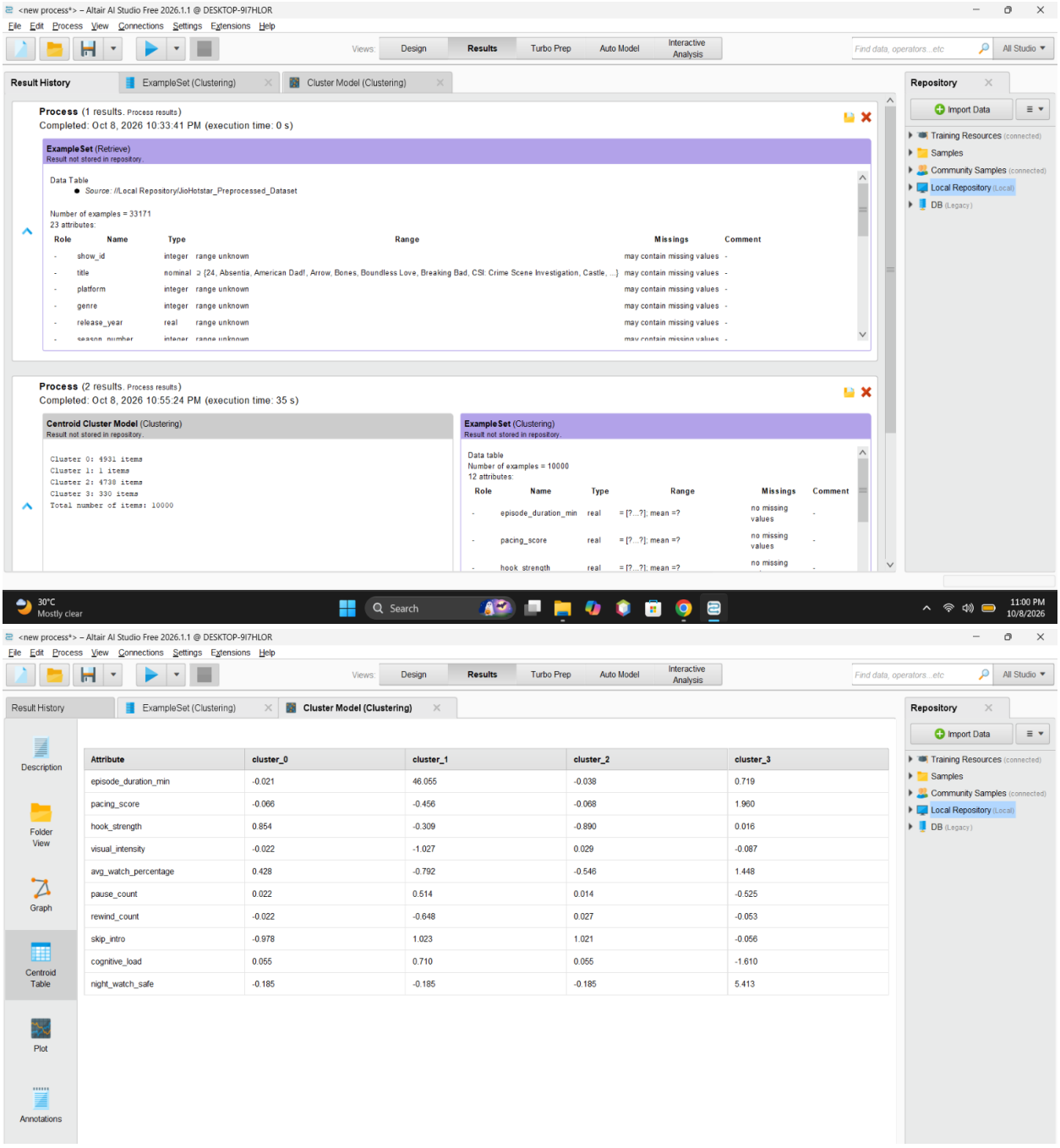


The visualization displays the clustering results using different colours to represent the clusters. Each plotted point represents a record, and the colours distinguish the cluster assignments. This visualization helps observe how records are distributed among the identified groups. The displayed graph uses pacing_score as the value column and cluster for colour. Since the X-axis is not assigned to a specific dataset attribute, this graph is mainly useful for viewing the distribution of cluster labels against the selected values.





The Statistics view displays information about the attributes in the clustered dataset, such as their data types, missing-value counts, minimum values, maximum values, averages, and deviations. The screenshot shows the distribution of the cluster attribute and statistics for numerical attributes such as episode duration, pacing score, and hook strength. This helps verify the processed data and understand its distribution. The normalized numerical attributes have averages close to zero and standard deviations close to one, as expected from Z-transformation.



The Result History displays the outputs generated after executing the process. It confirms that the original dataset was retrieved with 33,171 records and that the clustering output contains 10,000 records. The Cluster Model output also displays the number of records assigned to each cluster. The reduction to 10,000 records was due to the example limit in the free version of Altair AI Studio. This screenshot provides an overall confirmation of the execution and the generated results.

Metric / Aspect	Experiment 6: Python (scikit-learn K-Means)	Experiment 7: Altair AI Studio (K-Means)
Number of Clusters (k)	4	4
Dataset records processed	33,171	10,000
Cluster Counts	Exact counts not available in the retained results	Cluster 0: 4,931; Cluster 1: 1; Cluster 2: 4,738; Cluster 3: 330
Silhouette Score	Exact score not available	Not provided
Inertia	Exact value not available	Not provided
Davies–Bouldin Index	Exact value not available	Not provided
Preprocessing	StandardScaler	Replace Missing Values and Normalize using Z-transformation
Visualization	Cluster plots using selected engagement features	Scatter/Bubble plot coloured by cluster
Main Observation	Four groups were identified based on viewing and engagement characteristics	Most records belong to Clusters 0 and 2; Cluster 1 contains only one record

Comparison of Rapid Miner and Python

Feature	Python	RapidMiner
Customisation	High Flexibility .	Low (GUI based , pre-set options).
Ease of Use	Requires coding knowledge.	Easier for beginners.
Evaluation Metrics	Silhouette score used for validation.	No explicit evaluation metric.
VIsualisation	More customizable with Matplotlib/Seaborn.	Built in visualisation tools.

Conclusion:

Thus, K-Means clustering was successfully performed on the JioHotstar dataset using Altair AI Studio. The dataset was preprocessed by handling missing values and applying Z-transformation normalization before grouping the records into four clusters. Out of 10,000 processed records, most belonged to Clusters 0 and 2. The experiment helped me understand how clustering can group similar episodes based on viewing and engagement characteristics. One limitation was the free-version restriction of 10,000 records.

DMBI Laboratory - Experiment 7

Experiment 7: Clustering using RapidMiner / Altair AI Studio (K-Means Clustering)

Student Name: Atharva Waghmode

Roll Number: 59

Division: D15A

Course / Subject: Data Mining and Business Intelligence (DMBI)

Academic Year: 2026 - 2027

Official GitHub Experiment & Submission Links:

1. Individual GitHub Link for Experiment 7:

https://github.com/wagha-builds/dmbi_exps_59/tree/main/Experiment_7

2. Master GitHub Repository Link:

https://github.com/wagha-builds/dmbi_exps_59

Note: This document has been submitted as soft copy and printed as hard copy per lab guidelines. All supporting datasets, code files, and analytical outputs are maintained at the GitHub links above.